

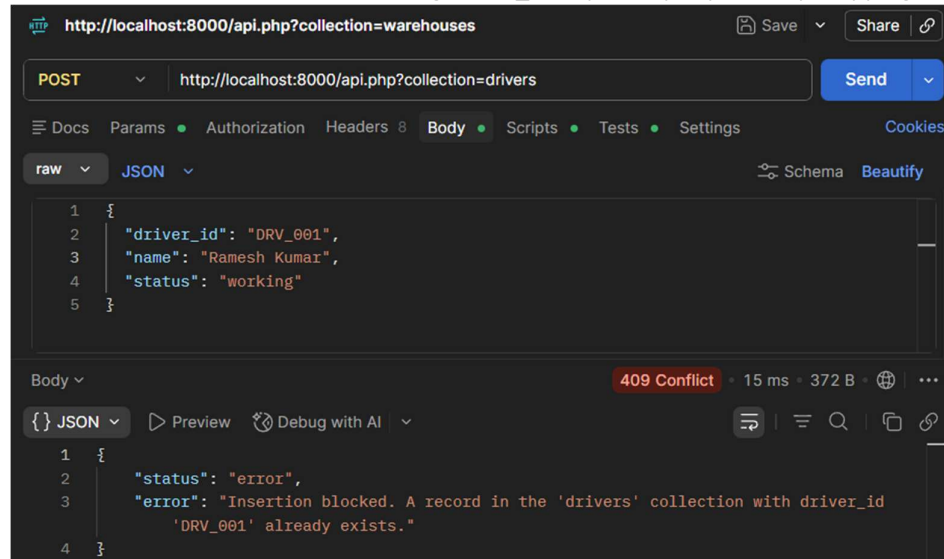
TESTING:

Test 1: The Duplicate ID Blocker (Primary Key Protection)

Defensive API Architecture: Duplicate Primary Key Prevention

Action: the seeder just created DRV_001, we will attempt to create it again.

Here, In drivers collection we are using driver_id as primary key field(By mapping to object iDs)

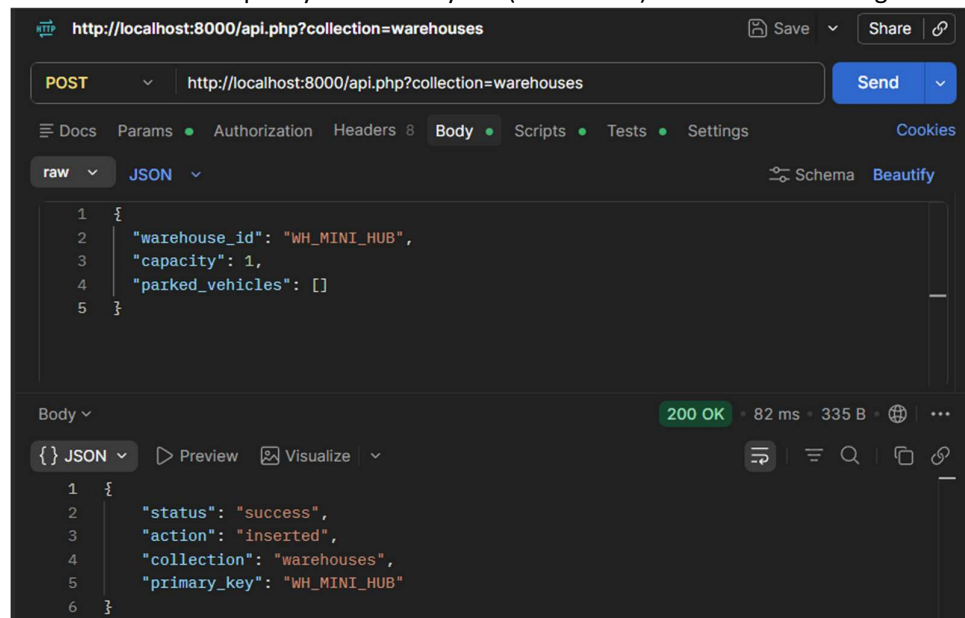


This test verifies the API's duplicate prevention mechanism. By mapping the custom `driver_id` directly to MongoDB's immutable `_id` primary key field, the database natively intercepts duplicate entries. As shown, the application successfully blocks the insertion and returns an HTTP 409 Conflict error, protecting the system from identical records.

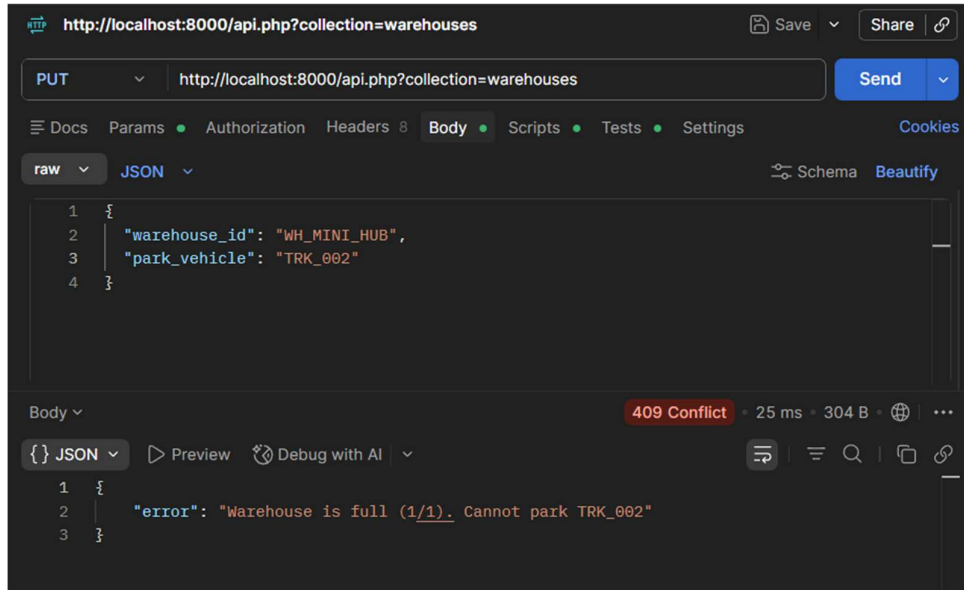
Test 2: Real-Time Array Capacity Validation

Dynamic One-to-Many Relationships: Capacity Overflow Protection

The Action: We will quickly create a tiny hub(warehouse) and overflow it using the seeded trucks.



Try to add another truck(TRK_002)



This test verifies real-time capacity validation within the database. By dynamically checking the length of the `parked_vehicles` array against the warehouse's capacity limit before executing an `$addToSet` command, the API successfully blocks overflow attempts. The resulting HTTP 409 Conflict response ensures the NoSQL architecture accurately reflects physical constraints.